

GALILEO Graph Databases Group Second Deadline Report (19 November 2025)

Francesco Pivotto 2158296

Giorgia Amato 2159999

Alessio Demo 2142885

November 17, 2025

Contents

1	Introduction	2
1.1	Context and Objective	2
1.2	Report Structure	2
2	Core Architectural Design and Execution Flow	3
2.1	Architectural Design Patterns for Modularity	3
2.2	Unified Architecture and Execution Workflow (The Pipeline)	4
2.3	Data Context and Output Standardization	4
2.3.1	Data Context Categorization	4
2.3.2	Standardized Output Format	5
3	Specific Baseline Implementation	6
3.1	Baseline NL (Natural Language)	6
3.2	Baseline SQL	6
3.3	Unified Command-Line Examples	6
4	Results and Evaluation	7
5	Future Work	8
6	Contributions	9

1 Introduction

1.1 Context and Objective

The preceding project phase involved the **assessment of query execution** on the database for each dataset, followed by the processing of the results obtained from our implementation using the `galois_eval.py` script.

The primary objective of the current task is to detail the **development process** implemented to replicate the following **baselines**, specifically utilizing the **Internal Knowledge datasets**:

- **SQL Baseline:** The system directly prompts the *Large Language Model* (LLM) with an **SQL query** as the input.
- **NL (Natural Language) Baseline:** The system employs a **Natural Language prompt** as the direct input to the LLM.

1.2 Report Structure

This introductory section establishes the methodological context of the project:

- Section 2 details the project’s core: the Unified Architecture and Execution Flow common to both baselines. This section describes the software design patterns adopted for modularity, the standardized execution pipeline and the definitions for the data context and output format (FULL JSON).
- Section 3 describes the specific implementations for the two evaluation modes, the NL Baseline and the SQL Baseline. Leveraging the common architecture, this section focuses on how each baseline handles its unique input (a natural language prompt or an SQL query) within the unified execution flow.
- Section 4 show, for each implemented baseline, **synoptic tables** reporting the derived evaluation measures and metrics. It analyzes and compares key metrics (F1-CELL, CARDINALITY, TUPLE-CONSTRAINT) and costs (tokens and latency) for both baselines across the different LLM providers tested.
- Section 5 outlines Future Work that will be made in the next tasks.
- Section 6 details the individual contributions of each team member to the task’s development.

2 Core Architectural Design and Execution Flow

2.1 Architectural Design Patterns for Modularity

The codebase’s central logic is structured around established software design patterns to ensure high modularity, extensibility, and maintainability across heterogeneous components.

Adapter Pattern (`llm_wrappers.py`): This pattern standardizes the interface for interacting with heterogeneous LLM providers (e.g., Gemini, Watsonx). By abstracting the provider-specific connection details, it enables the core execution engine to interact with all models through a uniform API.

Factory Pattern (`llm_factory.py`): Implemented as a simple factory method, this pattern decouples client components (e.g., `sql_baseline.py`, `nl_baseline.py`) from the specific instantiation logic of concrete LLM wrapper objects. Extensibility to new providers is achieved solely by updating the internal `PROVIDER_MAP` registry.

Template Method Pattern (`LLMBaseWrapper` in `llm_wrappers.py`): The abstract `LLMBaseWrapper` defines the invariant sequence (the `*template*`) for the life cycle of the LLM instance retrieval. It establishes the mandatory sequence while deferring the implementation of the provider-specific connection details (via `create_llm_instance()`) to concrete subclasses.

Chain/Pipeline Pattern (LCEL) (`build_lcel_chain.py`): This pattern, realized using LangChain Expression Language (LCEL), defines a structured, sequential execution flow:

Context Generation → Prompt Construction → LLM Invocation

This structured approach ensures predictable and auditable data transformation across the execution path.

2.2 Unified Architecture and Execution Workflow (The Pipeline)

To ensure high modularity, extensibility, and maintainability, we adopted a unified base architecture that serves both baselines (NL and SQL). This architecture centralizes the execution logic, abstracting interactions with LLM providers and standardizing the data flow.

The execution workflow is structured into a four-stage pipeline, ensuring robust and consistent processing across all baseline types:

1. **Context Information Retrieval:** The `get_db_context` function dynamically retrieves the necessary database context that will be attached to the LLM prompt together with the corresponding SQL query for SQL baseline or NL prompt for NL baseline. For instance this includes retrieving the database schema for SQL/NL baselines.
2. **Prompt Construction and Injection:** The derived context is programmatically injected into a sophisticated `ChatPromptTemplate`. This stage manages the insertion of system instructions and the baseline-specific input (the NL or SQL query), and also enforces the required JSON output structure via an `PydanticOutputParser` object.
3. **LLM Invocation:** The fully prepared and structured prompt is transmitted to the configured LLM instance, which was instantiated via the Factory/Adapter layers.
4. **Response Standardization:** The `parse_llm_response` function enforces cross-provider output standardization: This involves the mandatory validation and parsing of the requisite JSON structure returned by the LLM. It then enriches this output into the `FULL_JSON` format by extracting the `token consumption` from the LLM's response metadata and internally calculating and appending the `LLM response latency`.

This last standardized and enriched data process is essential for downstream performance evaluation, furthermore thanks to this centralized architecture, the execution logic is provider-independent, and the workflow is uniform.

2.3 Data Context and Output Standardization

This section defines the two primary contexts used to condition the Large Language Models (LLMs) and establishes the **standardized JSON output format** mandatory for all baseline evaluations (NL, SQL, and Palimpzest).

2.3.1 Data Context Categorization

The evaluation utilizes two distinct dataset contexts, governing the type of factual information provided to the LLM during prompting:

- **Internal Knowledge (IK):** In these datasets, the model relies exclusively on its pre-trained knowledge and linguistic reasoning capabilities. The prompt includes only the natural language question and the database schema for both the **SQL** and **NL** baselines, but no factual data retrieved from the database.

- **Model Context (MC):** These datasets are designed to enrich the prompt with external information retrieved from a knowledge base. For this specific project task, MC datasets were **not used** in either the NL or SQL baselines, but could represent a good resource for RAG purposes.

2.3.2 Standardized Output Format

Every response produced by any baseline is written in a JSON file that fits the mandatory **FULL JSON** format. This rigorous structure ensures consistent computation of metrics across all providers and baselines.

```
{
  "result_set": [
    { "originaltitle": "The Three Musketeers"},
    { "originaltitle": "The Count of Monte Cristo"}
  ],
  "time": 1.84,
  "tokens": 312
}
```

Figure 1: FULL JSON Output Format Example

Field Name	Description
result_set	Denotes the structured LLM output. This must be an array of JSON objects corresponding to the query's selected columns.
time	Represents the total time taken by the LLM to generate the response.
tokens	Indicates the number of tokens processed during the interaction.

3 Specific Baseline Implementation

Leveraging the unified architecture described in Section 2.2, the two baselines differ almost exclusively in the type of input provided to the LLM during the "Prompt Construction" phase.

3.1 Baseline NL (Natural Language)

The objective of this baseline is to evaluate the model's intrinsic reasoning capabilities starting from natural language.

- **Input:** Natural language prompts are loaded from dataset-specific resource files (e.g., `queries_<dataset-name>.txt`).
- **Prompt Logic:** The user input (HUMAN PROMPT) is composed of the **natural language question** and the database schema information.

3.2 Baseline SQL

The objective of this baseline is to assess the LLM's capacity to interpret complex SQL syntax, perform logical reasoning over the schema, and produce a structured output.

- **Input:** SQL queries are systematically loaded from the (e.g., `queries_<dataset-name>.sql`) files.
- **Prompt Logic:** The user input (HUMAN PROMPT) is populated with the **SQL statement** itself, with the database schema added as additional knowledge.

3.3 Unified Command-Line Examples

Only for knowledge we want to show what commands and what params the system accepts from the CLI for running the baselines:

Argument	Type	Function
<code>datasets</code>	Positional	Accepts zero to multiple dataset identifiers (e.g., <code>WORLD</code> , <code>GEO</code>).
<code>-mode</code>	Option	Selects the execution type: <code>nl</code> , <code>sql</code> or <code>both</code> .
<code>-provider</code>	Option	Overrides the default LLM provider defined in the configuration (<code>gemini</code> or <code>watsonx</code>).

High-Level Logical Flow The overall execution sequence is dictated by the input received from the CLI:

1. **Initialization:** Argument parsing is performed to determine the list of target datasets, prioritizing the CLI input over the default configuration settings.
2. **Conditional Invocation:** Based on the selected `-mode`, the system conditionally invokes the NL, SQL or both.
3. **Task Dispatch and Persistence:** The task is dispatched to the provider-agnostic implementation, and the resulting `FULL_JSON` output files are persistently stored for evaluation.

4 Results and Evaluation

The final quantitative outcomes for the SQL Baseline are integrated with the NL results, summarized in tables detailing the **F1-CELL**, **CARDINALITY**, and **TUPLE-CONSTRAINT** metrics, along with their average scores, average inference time (**latency**), and total token consumption.

The results obtained from the Baseline NL and SQL baseline modules were evaluated using standard metrics to assess the performance of the LLMs in both Natural Language (NL) and SQL contexts. The model used for the **gemini** provider is: **gemini-2.5-flash**, instead for **IBMWatsonx** provider we used **meta-llama/llama-3-3-70b-instruct**.

IMPORTANT: We want to specify that we tested the baselines using either Gemini LLM and IBM watsonx: For the former in particular was necessary to handle the free-plan requests for minute limitation, for this reason the running time of the baselines with gemini could be more longer that watsonx. For this purpose we implemented a simple backoff retry mechanism that in case of 'out of quota' error response from the gemini API waits for some time before retrying the request. All systems are tested on the datasets described in Table 2 of the previous project task, excluding Premier and Fortune, as the Galois authors did.

Metric	NL	SQL
F1-CELL	0.435	0.257
CARDINALITY	0.741	0.571
TUPLE-CONSTR.	0.282	0.119
#TOKENS(M)	0.930	0.063
AVG-TIME (s)	9.050	4.858
AVG-SCORE	0.486	0.315

Table 1: GEMINI results

Metric	NL	SQL
F1-CELL	0.436	0.523
CARDINALITY	0.701	0.718
TUPLE-CONSTR.	0.212	0.243
#TOKENS(M)	0.015	0.012
AVG-TIME (s)	5.353	4.485
AVG-SCORE	0.450	0.495

Table 2: WATSONX results

Some comments about the results in the tables above:

The analysis of results across different runs reveals that quality and cost metrics are intrinsically linked to the precision of the underlying Large Language Model (LLM) (various models have different reasoning ability and prompt sensibility as you can see in the result metrics reported above) this is one of the reasons for which LLM's performance can fluctuate, impacting key metrics. Another trivial factor is the LLM's tendency to generate values or parameter names that are semantically similar but syntactically different from the expected attribute names or those reported in the ground truth JSON files, this discrepancy, even if minor, can lead to false negatives.

Finally, variability is reflected in costs: the number of tokens consumed (#TOKENS(M)) and the average execution time (AVG-TIME (s)) can vary significantly across providers, depending on different internal prompting strategies and context limitations.

5 Future Work

For next tasks, we will consider improving and extending the actual logical and structural retrieval information process: In particular will focus on the integration and deployment of the GALOIS framework to significantly improve the accuracy and efficiency of structured data retrieval. This integration is crucial for addressing the limitations inherent in directly prompting Large Language Models (LLMs) with complex Natural Language (NL) or SQL queries, which often result in low precision and recall.

Our implementation effort will concentrate on two main areas: **Logical Plan Optimization** and **Physical Operator Design**.

- **Logical Plan Optimization:** We will implement the **Filter-LLMScan** operator, which allows selection conditions to be pushed down directly to the LLM. This is a critical step because the choice of which conditions to push is complex; pushing down all conditions can reduce token cost but often degrades result quality due to the LLM’s difficulty in processing complex requests.
- **Physical Operator Design:** We will implement and evaluate both alternative physical scan operators supported by GALOIS:
 - **Table-Scan:** A single prompt requests the LLM to return the entire result set in a structured JSON format. This approach is token-efficient but may produce lower quality results.
 - **Key-Scan:** This two-step operator first retrieves the key values for the relevant tuples and then iteratively queries the LLM to populate the remaining attribute values for each key.

Summarizing the meaning: to decide between these two strategies described above, we will incorporate GALOIS’s confidence threshold (τ) mechanism¹⁶. By computing the confidence of the LLM in retrieving key values, we can select Key-Scan when confidence is high (improving quality) or revert to Table-Scan when confidence is low (for more reliable, albeit potentially less complete, data retrieval).

Furthermore we have implemented also a RAG system that follows the following steps:

- Read a specific folder containing text files (like .md, .txt, or files with no extension).
- Process these files by breaking them into small chunks and converting those chunks into numerical vectors (embeddings) using a Hugging Face model.
- Create an in-memory vector database (using FAISS) to allow for fast searching.
- When it receives a prompt, it searches that database and retrieves the chunks of text that are most relevant to the question.

We did this to have a solid base in case we need it for completing future tasks.

6 Contributions

Alessio Demo (Badge number: 2142885)

- **First Deadline:** Developed function database connection and instance creation of DuckDB database manager for each dataset using the ingest files (.duckdb files). Developed function for retrieving the SQL queries from .sql files, processing the queries and executing them on DuckDB database instance and finally store it in a json file.
- **Second Deadline:** Implemented the Baseline NL function for interacting with the LLM, save the results in FULL JSON format and next using them in the main script. Unit tests for the baseline_tools script using pytest. Palimpsest baseline implementation using external libraries for creating the embeddings and evaluate the similarity between them in relation to a given prompt.

Writing the reports sections reporting the complete pipeline followed for developing the stuff above.

Francesco Pivotto (Badge number: 2158296)

- **First Deadline:** Implemented a modular project structure by relocating core logic to `src/` and consolidating configuration and database management logic within dedicated utility modules (`config/loaders.py`). Established the framework for multi-provider support by adding initial integration for the Grok LLM, including dedicated configuration points (`.env`, `config.yaml`, `loaders.py`).
- **Second Deadline:** Completed initial implementation of the SQL baseline, including the core logic for connecting to DuckDB and structuring query execution via the Watsonx provider. Fixed critical issues in environment variable loading, ensuring robust and standardized retrieval of secrets and settings via the central configuration object (using `loaders.py`). Modularized the `sql_baseline` module, incorporating the LCEL Pattern to prepare for dynamic LLM selection. Centralized general utility functions by moving helpers into `baseline_tools`. Introduced the dedicated `llm_wrapper` class to handle all LLM connection logic, API parameter management, and calls.

Giorgia Amato (Badge number: 2159999)

- **First Deadline:** Contributed to project initialization by setting up the repository structure, environment configuration and utility scripts. Added EXPLAIN and EXPLAIN ANALYZE support with JSON serialization, implemented JSON generation for query outputs, and improved logging and test utilities.
- **Second Deadline:** Developed the Gemini-based SQL baseline, including IK/MC query handling, schema and raw-data injection into prompts, JSON parsing through Pydantic, and error/timeout management. Refactored and extended unit tests for baseline tools and added new tests validating LLM interaction and Gemini SQL execution.